

Translating Environments with Coding Agents

Ada Lindqvist

Tomas Okafor

Abstract

Environments for reinforcement learning are slow, and rewriting them by hand takes months. We have a coding agent translate them instead, and check each translation at four levels, from single functions to policies trained in one and evaluated in the other.

1 Introduction

Simulating the environment takes most of the time spent training an agent. Rewriting the environment for an accelerator takes a specialist months, and has been done for a handful of them. We let a coding agent do the rewriting, and check what it writes at four levels of increasing scope, so that the agent is told where a translation is wrong as well as that it is wrong. This paragraph runs on for a few lines so that the page has a measure of its body text, as a real paper’s would.

The translations are as fast as the hand-written rewrites they are compared with, and pass the same tests. They cost a few dollars each, and the agent wrote every line of them.

A Supplementary Details

A.1 Multi-Agent Validation

We translated two of the environments again with two more agents, using the same prompts and the same tests. Both converge to translations that pass the tests (Table ??), so the method does not depend on the agent.

Table 1: **Multi-agent comparison.** Identical inputs; equivalent outputs.

Environment	Agent	Iters	Tests	Cost
Pong	Agent One	13	6/6	\$0.05
	Agent Two	3	5/6 [§]	\$0.08
HalfCheetah	Agent One	20	69/69	\$3.26
	Agent Three	6	69/69	— [†]

[§]Same statistical test for both agents. [†]Cost not tracked.

A.2 Verification Ablation Details

HalfCheetah (complex physics). The run checked only by rollouts used eight end-to-end tests and forty-two iterations without converging. The agent could not tell a sign error in a force from a failure at the end of a rollout, and went round rewriting the vectorization instead.

A.3 Cross-Hardware Validation

Table 2: **Throughput elsewhere.** Peak steps per second.

Environment	SPS	vs. Reference
HalfCheetah	13.1M	290×
Pong	1.4B	23k×

A.4 Agent Translation Metrics

The translation was carried out entirely through logged sessions. The first phase translated five core modules, and the second used a command-line agent for the six modules heaviest in logic.

B Performance Optimization Guide

The following patterns, distilled from the case studies, make an environment faster. They are ordered by how much they usually help.

- 1. Fixed-size state arrays.** The compiler needs every shape known in advance. Replace all dynamic-length data structures (lists, dictionaries with varying keys) with fixed-size `jnp.ndarray` fields padded to their largest size, and mark the unused slots with a sentinel value.
- 2. Branchless conditionals with `jnp.where`.** Replace Python `if/else` with `jnp.where(condition, true_val, false_val)`, which computes both branches and selects the result by mask; this is faster on a GPU because it avoids divergence:

```
ball_vy = jnp.where(wall_hit, -ball_vy, ball_vy)
step = jax.vmap(
    partial(step, terrain=terrain),
    in_axes=(0, 0)
)
```

- 3. JIT the outer interface.** Apply `jax.jit` to the vectorized step and reset functions so that the whole batch is compiled to a single kernel, and compile them once while the environment is set up.

B.1 Optimization Agent Prompt

The following prompt is used once the environment passes the tests. It asks the agent to make it faster without changing what it does.

```
The environment has passed all verification tests. Now optimize it
for maximum steps per second.
```

```
Target: Maximize SPS while keeping every test passing.
```

```
Constraints:
```

- All tests must continue to pass after optimization
- Do not change the external API (step, reset, observation shapes)

Algorithm 1 Hierarchical translation and verification.

```
1: Phase 1: Module translation
2: for  $k = 1$  to  $K$  do
3:    $m'_k \leftarrow \text{AGENT}(m_k)$ 
4:   if  $\text{RUNTESTS}(m'_k) = \text{PASS}$  then
5:     break
6:   end if
7: end for
8: return  $E_{\text{perf}}$ 
```

Algorithm 1 sets out the loop the agent runs, module by module, until every test passes.